

Create a custom pull connector

- Release version: Australia
-
- Updated March 12, 2026
-
- ⌚ 4 minutes to read

You can create a customized pull connector that requires a script, connector definition, and connector instance, to retrieve events on behalf of an event source.

Before you begin

Role required: evt_mgmt_admin

About this task

A custom connector script can make remote API calls for a new event source to send events to the ServiceNow instance. You can create a script, and then add it as part of a new Event Management connector definition and connector instance. This procedure automatically creates a JavaScript code file using the name that you enter in the Name field as the filename. The JavaScript custom code that accomplishes these actions must:

- Connect to an event monitoring tool.
- Retrieve events from an event monitoring tool.
- Send events to the Event [em_event] table using a web service API. See [REST API](#).

Each connector definition is specific to an event source vendor. The connector definition specifies the MID Server script include that pulls events from the external event source. In addition, the connector definition specifies what connector instance value parameters are needed to connect to the external event source host.

Procedure

1. Create a custom MID Server script include.

This example uses Groovy (deprecated). In place of a script include in Groovy, it is recommended to use JavaScript.

- a. Implement these methods:

- @Override OperationStatus testConnection()
- @Override OperationStatus execute()

- b. Design the class to extend the ThirdPartyConnector.

```
public class HypericConnector extends ThirdPartyConnector
```

- c. Import platform classes for event creation, sending, logging, and third-party connector base classes.

```
package com.service_now.mid.probe.tpcon.test

import com.glide.util.Log
import com.service_now.mid.MIDServer
import com.service_now.mid.probe.event.IEventSender
import com.service_now.mid.probe.event.SNEventSender
import com.service_now.mid.probe.tpcon.OperationStatus
import com.service_now.mid.probe.tpcon.OperationStatusType
import com.service_now.mid.probe.tpcon.ThirdPartyConnector
import com.snc.commons.eventmgmt.Event
```

- d. Connect to the data collector, such as SCOM or VMware vRealize Hyperic (Hyperic), with the API provided by the collector.
- e. Use the context parameters to set the event field values that came from a connector instance.

```
def authString = (context.username + ':' + context.password).getBytes()
def urlStr = 'http://' + context.host + ':' + context.parameters.port +
```

- f. Implement the execute function in the script.

It reads events from the connector using its API to create Event objects and send them to the ServiceNow instance. For example:

```
GPathResult alertsResponse1 = readAlerts('hqu/hqapi1/alert/find.hqu?k
'&end=' + tillStr + '&count=1000000&severity=1&inEscalation=false&notFi
Event event = new Event()
event.emsSystem = context.name
event.source = "Hyperic"
event.description = it.@reason
event.type = it.@name
...
IEventSender eventSender = MIDServer.getSingleton(SNEventSender.class)
for (Event event : list) {
    eventSender.sendEvent(event)
}
```

2. Navigate to MID Server > MID Server Script Files and create a script.
3. Specify the Parent field as Groovy, complete the form as appropriate, and then click Submit.
4. Navigate to Event Management > Integrations > Connector Definitions and create a connector definition.
5. In the Groovy script to run field, select the MID Server script file, complete the form as appropriate.
In addition to username or host, you can add any other parameter, for example, port, and then click Submit.
6. Navigate to Event Management > Integrations > Connector Instances and create a connector instance.
7. In the Connector definition field, select the connector definition, complete the form as appropriate, and then click Submit.
8. To confirm or debug the script, use debug printouts from Groovy to the MID Server log.
9. To monitor incoming events using the custom connector instance, navigate to ECC > Queue and filter on ConnectorProbe.
10. Create a custom connector definition.
 - a. Navigate to All > Event Management > Integrations > Connector Definitions.
 - b. Click New.
 - c. In the Name field, type a descriptive name for the connector definition.
 - d. In the Script type field, select Javascript.
 - e. Right-click the form header and select Save.

The Javascript to run field is automatically populated with the name of the JavaScript template file.
11. To edit the default JavaScript file, click the information icon () to the right of the JavaScript file name and follow these steps:
 - a. Specify the information required for the testConnection function.
This function tests the connector definition to verify if the connection to the target is valid.
 - b. Specify the information required for the execute function.
This function retrieves the information from the external source.
 - c. Specify the information required for the retrieveKpi function.
This function retrieves the metric data from the external source.
12. **Optional:** You can replace the default JavaScript with your JavaScript code.
13. Fill in the fields, as appropriate.

Table 1. Connector Definition form

Field	Description
Default schedule	The number of seconds between attempts to receive events.
Bi-directional	Specify to enable the external monitoring system to be updated if the alert is changed. There is default implementation for SCOM.

Field	Description
Alert field identifier	Specify the alert field that, if changed manually, causes the external monitoring system to be updated. Note: This field appears only if Bi-directional is selected.
Collect metrics	Select to enable the collection of metrics. Note: Metrics are collected only for SCOM external event sources.
Metrics collection default schedule (seconds)	Specify the number of seconds of information that must be collected. The default is 10 seconds. Note: This field appears only if Collect metrics is selected.
Connector Parameters	Specify the parameters to enable communication with an event server.
Connector Definition to MID Server Capabilities	The MID Server name to process events from the event server. If not specified, an available MID Server is used.

14. Click Submit.